

# RIDEFRIEND

## Arquitectura Multi-Mercado

Uma Codebase · Seis Mercados · Localização Automática

AO MZ BR CV PT NG

|        |            |        |            |          |         |
|--------|------------|--------|------------|----------|---------|
| Angola | Moçambique | Brasil | Cabo Verde | Portugal | Nigéria |
|--------|------------|--------|------------|----------|---------|

|                       |                                                        |
|-----------------------|--------------------------------------------------------|
| Conceito Central      | Internacionalização (i18n) + Configuração por Mercado  |
| Padrão Arquitectural  | Market Config + i18n Namespace + Feature Flags         |
| Linguagens Suportadas | Português AO/PT/MZ/CV + Português BR + Inglês NG       |
| SMS Gateway           | AfricasTalking (África) + Twilio (Brasil/Portugal)     |
| Mapas                 | OpenStreetMap (todos) + Google Maps (BR/PT opcional)   |
| Pagamentos            | Por mercado: Multicaixa, M-Pesa, PIX, MB Way, Paystack |
| Versão do Documento   | 1.0 · 2025                                             |

# 1. O Princípio: Uma Codebase, Muitos Mercados

A abordagem correcta não é criar uma cópia da app por país. Isso significa manter seis bugs em vez de um, lançar seis actualizações em vez de uma, e duplicar todo o trabalho para sempre. A solução é construir um sistema de configuração por mercado desde o primeiro dia — uma arquitectura onde o código é universal e os dados que mudam (idioma, moeda, operadoras, estilos de mapa, termos locais) são carregados dinamicamente conforme o mercado detectado.

*Regra de Ouro: se algo muda de mercado para mercado, é uma CONFIGURAÇÃO, não código. O código executa a lógica — a configuração define o comportamento local.*

## 1.1 O que muda por mercado

| Dimensão                | Tipo de Mudança | Exemplos                                                                                            |
|-------------------------|-----------------|-----------------------------------------------------------------------------------------------------|
| Linguagem               | i18n + dialecto | AO/PT/MZ: "boleia". BR: "carona". NG: "lift". AO: "paragem" / PT: "paragem" / BR: "ponto de ônibus" |
| Vocabulário técnico     | i18n namespace  | PT: "Condutor" / AO+BR: "Motorista". BR: "Estou aqui!" vs AO: "Agora!"                              |
| Moeda                   | Market config   | AOA (Kz) / MZN (MT) / BRL (R\$) / EUR (€) / NGN (₦) / CVE (Esc)                                     |
| Formato de telefone     | Market config   | +244 (AO) / +258 (MZ) / +55 (BR) / +238 (CV) / +351 (PT) / +234 (NG)                                |
| Gateway SMS OTP         | Market config   | AfricasTalking (África) / Twilio (BR, PT) / Termii (NG)                                             |
| Mapas e geocoding       | Market config   | OSM+Nominatim (todos). Google Maps opcional para BR/PT (maior cobertura)                            |
| Paragens de referência  | Base de dados   | Coordenadas de paragens pré-carregadas por cidade                                                   |
| Pagamentos in-app       | Feature flag    | Multicaixa/EMIS (AO), M-Pesa (MZ), PIX (BR), MB Way (PT), Paystack (NG)                             |
| Regulação/Privacidade   | Feature flag    | RGPD obrigatório PT. LGPD obrigatório BR. Dados podem sair do país? (AO: sim)                       |
| Horário e fuso          | Market config   | Africa/Luanda (WAT +1), Africa/Maputo (CAT +2), America/Sao_Paulo (BRT -3)                          |
| Estilo visual           | Market config   | Cores de destaque ligeiramente ajustadas. PT mais sóbrio, NG mais vibrante                          |
| Contactos de emergência | Market config   | PT: 112. AO: 113. BR: 190. NG: 199. MZ: 119                                                         |

## 1.2 O que NÃO muda — o código universal

- Algoritmo Haversine de cálculo de distância
- Lógica de detecção de passageiros na rota (isPointNearRoute)

- Sistema de ratings e reputação
- Fluxo de confirmação de boleia (estados: requested → accepted → completed)
- Sistema de notificações push (Expo Notifications)
- Lógica de SOS (só muda o número de emergência — via config)
- Base de dados Supabase (esquema idêntico, dados separados por market\_code)
- Componentes de UI (só cores e textos mudam — via theme e i18n)
- Toda a navegação e estrutura de ecrãs
- Autenticação OTP (o provider SMS muda — o fluxo não)

## 2. Arquitectura de Localização

### 2.1 Estrutura de Ficheiros

src/i18n/ e src/config/ — estrutura de pastas

```
src/
├── i18n/
│   ├── index.ts           ← configuração i18next
│   ├── detector.ts        ← detecta idioma/mercado automaticamente
│   └── locales/
│       ├── pt-AO/         ← Angola (base)
│       │   ├── common.json ← termos gerais
│       │   ├── home.json  ← ecrã principal
│       │   ├── auth.json  ← autenticação
│       │   ├── ride.json  ← termos de boleia
│       │   └── errors.json ← mensagens de erro
│       ├── pt-MZ/         ← só sobreescreve diferenças de Moçambique
│       ├── pt-BR/         ← diferenças Brasil (carona, ônibus...)
│       ├── pt-CV/         ← diferenças Cabo Verde
│       ├── pt-PT/         ← diferenças Portugal (condutor, RGPD...)
│       └── en-NG/         ← inglês Nigéria (ficheiros completos)
├── config/
│   ├── markets/           ← configuração por mercado
│   │   ├── index.ts       ← carrega config do mercado activo
│   │   ├── ao.config.ts   ← Angola
│   │   ├── mz.config.ts   ← Moçambique
│   │   ├── br.config.ts   ← Brasil
│   │   ├── cv.config.ts   ← Cabo Verde
│   │   ├── pt.config.ts   ← Portugal
│   │   └── ng.config.ts   ← Nigéria
│   └── features.ts        ← feature flags por mercado
└── hooks/
    ├── useMarket.ts       ← hook principal de acesso à config
    └── useTranslation.ts   ← wrapper do i18next com autocomplete TS
```

### 2.2 Interface do MarketConfig — o contrato

src/config/markets/types.ts

```
// Ficheiro: types.ts | Função: Interface de configuração por mercado

export interface MarketConfig {
  // Identidade
  code: MarketCode           // "ao" | "mz" | "br" | "cv" | "pt" | "ng"
  name: string               // Nome do mercado para exibição
  locale: string              // "pt-AO" | "pt-BR" | "en-NG" etc.
  timezone: string           // "Africa/Luanda" | "America/Sao_Paulo"

  // Telefone
```

```
phonePrefix: string          // "+244" | "+55" etc.
phoneMinDigits: number       // mínimo de dígitos após prefixo
phoneMaxDigits: number
phonePlaceholder: string     // "9XX XXX XXX" (AO) | "(11) 9XXXX-XXXX"
(BR)

// SMS OTP
smsProvider: "africas_talking" | "twilio" | "termii"
smsFromName: string          // nome que aparece no SMS

// Mapas
mapsProvider: "osm" | "google"
defaultCenter: { lat: number; lng: number }
defaultZoom: number
geocodingProvider: "nominatim" | "google"

// Moeda
currency: { code: string; symbol: string; decimals: number }
// ex: { code: "AOA", symbol: "Kz", decimals: 0 }

// Segurança
emergencyNumber: string      // "113" | "190" | "112" | "199"
privacyRegulation: "lgpd" | "rgpd" | "none"
dataResidency: "local" | "eu" | "any"

// Pagamentos (opcional por mercado)
paymentMethods?: PaymentMethod[]

// Feature flags
features: {
  inAppPayments: boolean     // pagamento de boleia in-app
  multiLanguageUI: boolean   // mais de um idioma na app
  googleMaps: boolean        // usar Google Maps em vez de OSM
  backgroundLocation: boolean // rastreamento em background
  vehicleVerification: boolean // verificação de matrícula/carta
  gdprConsent: boolean       // ecrã de consentimento RGPD
}

// Estilo visual
theme: {
  accentColor: string        // cor de destaque do mercado
  accentColorLight: string
}

// Suporte
supportWhatsApp?: string     // número WhatsApp do suporte local
appStoreId?: string         // para links de review
}

export type MarketCode = 'ao' | 'mz' | 'br' | 'cv' | 'pt' | 'ng';
```

## 2.3 Config por mercado — exemplo Angola e Brasil

```
src/config/markets/ao.config.ts (Angola — base)

// Ficheiro: ao.config.ts | Função: Configuração do mercado Angola
import type { MarketConfig } from './types';

export const aoConfig: MarketConfig = {
  code: 'ao',
  name: 'Angola',
  locale: 'pt-AO',
  timezone: 'Africa/Luanda',

  phonePrefix: '+244',
  phoneMinDigits: 9,
  phoneMaxDigits: 9,
  phonePlaceholder: '9XX XXX XXX',

  smsProvider: 'africas_talking',
  smsFromName: 'RideFriend',

  mapsProvider: 'osm',
  defaultCenter: { lat: -8.8390, lng: 13.2894 }, // Luanda centro
  defaultZoom: 13,
  geocodingProvider: 'nominatim',

  currency: { code: 'AOA', symbol: 'Kz', decimals: 0 },

  emergencyNumber: '113',
  privacyRegulation: 'none',
  dataResidency: 'any',

  features: {
    inAppPayments: false,
    multiLanguageUI: false,
    googleMaps: false,
    backgroundLocation: true,
    vehicleVerification: false,
    gdprConsent: false,
  },

  theme: { accentColor: '#10B981', accentColorLight: '#D1FAE5' },
  supportWhatsApp: '+244923000000',
};
```

```
src/config/markets/br.config.ts (Brasil — diferenças)

// Ficheiro: br.config.ts | Função: Configuração do mercado Brasil
import type { MarketConfig } from './types';

export const brConfig: MarketConfig = {
  code: 'br',
```

```

name: 'Brasil',
locale: 'pt-BR',
timezone: 'America/Sao_Paulo',

phonePrefix: '+55',
phoneMinDigits: 10,
phoneMaxDigits: 11,
phonePlaceholder: '(11) 9XXXX-XXXX',

smsProvider: 'twilio', // AfricasTalking não cobre Brasil
smsFromName: 'RideFriend',

mapsProvider: 'google', // Melhor cobertura em periferias brasileiras
defaultCenter: { lat: -23.5505, lng: -46.6333 }, // São Paulo
defaultZoom: 13,
geocodingProvider: 'google',

currency: { code: 'BRL', symbol: 'R$', decimals: 2 },

emergencyNumber: '190', // Polícia Militar Brasil
privacyRegulation: 'lgpd',
dataResidency: 'local', // LGPD: dados no Brasil

features: {
  inAppPayments: true, // PIX activo no Brasil
  multiLanguageUI: false,
  googleMaps: true,
  backgroundLocation: true,
  vehicleVerification: true, // DETRAN disponível
  gdprConsent: false,
},

theme: { accentColor: '#EAB308', accentColorLight: '#FEF9C3' },
paymentMethods: [{ id: 'pix', name: 'PIX', icon: 'pix' }],
};

```

## 2.4 Detector automático de mercado

```

src/il8n/detector.ts

// Ficheiro: detector.ts | Função: Detecta o mercado a usar na app
import * as Localization from 'expo-localization';
import AsyncStorage from '@react-native-async-storage/async-storage';
import type { MarketCode } from '@config/markets/types';

// Mapeamento de prefixo de telefone → mercado
const PREFIX_TO_MARKET: Record<string, MarketCode> = {
  '+244': 'ao', // Angola
  '+258': 'mz', // Moçambique
  '+55': 'br', // Brasil
  '+238': 'cv', // Cabo Verde

```

```
'+351': 'pt', // Portugal
'+234': 'ng', // Nigéria
};

// Mapeamento de locale do dispositivo → mercado
const LOCALE_TO_MARKET: Record<string, MarketCode> = {
  'pt-AO': 'ao', 'pt-MZ': 'mz', 'pt-BR': 'br',
  'pt-CV': 'cv', 'pt-PT': 'pt', 'en-NG': 'ng', 'en-GB': 'ng',
};

export async function detectMarket(): Promise<MarketCode> {
  // 1. Mercado guardado pelo utilizador (mais alta prioridade)
  const saved = await AsyncStorage.getItem('rf_market');
  if (saved) return saved as MarketCode;

  // 2. Locale do dispositivo (ex: pt-BR, pt-AO)
  const deviceLocale = Localization.locale; // ex: "pt-BR"
  if (LOCALE_TO_MARKET[deviceLocale]) return LOCALE_TO_MARKET[deviceLocale];

  // 3. Fallback: parte do locale (ex: pt-BR → pt)
  const lang = deviceLocale.split("-")[0];
  if (lang === 'pt') return 'ao'; // padrão português = Angola
  if (lang === 'en') return 'ng'; // padrão inglês = Nigéria

  return 'ao'; // fallback absoluto
}

export async function setMarket(code: MarketCode): Promise<void> {
  await AsyncStorage.setItem('rf_market', code);
}
```



## 3. Sistema de Tradução (i18n)

### 3.1 Filosofia — Apenas as Diferenças

Cada mercado lusófono herda as traduções de Angola (base pt-AO) e sobrescreve apenas as palavras diferentes. Assim, se uma string não existir em pt-MZ, a app usa automaticamente a versão pt-AO. Só a Nigéria tem ficheiros completamente separados em inglês.

*Princípio de herança: pt-BR herda de pt-AO. pt-MZ herda de pt-AO. pt-CV herda de pt-AO. pt-PT herda de pt-AO. Cada mercado só define as suas diferenças — sem duplicação.*

### 3.2 Ficheiros de tradução — base Angola e diferenças

```
src/i18n/locales/pt-AO/ride.json (base - Angola)

{
  "request_ride": "Pedir Boleia",
  "offer_ride": "Oferecer Boleia",
  "ride_confirmed": "Boleia Confirmada!",
  "ride_cancelled": "Boleia cancelada",
  "driver_approaching": "{{name}} está a aproximar-se",
  "im_here_btn": "Estou Aqui! · Avisar Rede",
  "im_here_sent": "Rede Alertada! · {{count}} contactos notificados",
  "stop_label": "Paragem",
  "waiting_since": "À espera há {{time}}",
  "driver_label": "Motorista",
  "passenger_label": "Passageiro",
  "nearby_drivers": "Motoristas Conhecidos",
  "no_drivers": "Nenhum motorista conhecido próximo.",
  "eta_now": "Agora!",
  "eta_min": "{{n}} min",
  "distance_m": "{{n}} m",
  "distance_km": "{{n}} km",
  "at_stop": "Na paragem"
}
```

```
src/i18n/locales/pt-BR/ride.json (Brasil - só diferenças)

{
  "request_ride": "Pedir Carona",
  "offer_ride": "Oferecer Carona",
  "ride_confirmed": "Carona Confirmada!",
  "ride_cancelled": "Carona cancelada",
  "driver_approaching": "{{name}} está chegando",
  "im_here_btn": "Tô Aqui! · Avisar a Galera",
  "im_here_sent": "Galera avisada! · {{count}} contatos notificados",
  "stop_label": "Ponto de Ônibus",
  "waiting_since": "Esperando há {{time}}",
  "driver_label": "Motorista",
  "passenger_label": "Passageiro",
}
```

```

    "nearby_drivers": "Motoristas Conhecidos",
    "no_drivers": "Nenhum motorista conhecido por perto.",
    "eta_now": "Chegando!",
    "at_stop": "No ponto"
    // Restantes strings herdadas de pt-AO
  }

```

src/i18n/locales/en-NG/ride.json (Nigéria — inglês completo)

```

{
  "request_ride": "Request a Lift",
  "offer_ride": "Offer a Lift",
  "ride_confirmed": "Lift Confirmed!",
  "ride_cancelled": "Lift cancelled",
  "driver_approaching": "{{name}} is on the way",
  "im_here_btn": "I'm Here! · Alert My Network",
  "im_here_sent": "Network alerted! · {{count}} contacts notified",
  "stop_label": "Bus Stop",
  "waiting_since": "Waiting for {{time}}",
  "driver_label": "Driver",
  "passenger_label": "Passenger",
  "nearby_drivers": "Nearby Drivers",
  "no_drivers": "No known drivers nearby.",
  "eta_now": "Arriving now!",
  "eta_min": "{{n}} min",
  "distance_m": "{{n}} m",
  "distance_km": "{{n}} km",
  "at_stop": "At the bus stop"
}

```

src/i18n/locales/pt-PT/ride.json (Portugal — diferenças)

```

{
  "driver_label": "Condutor",
  "nearby_drivers": "Condutores Conhecidos",
  "no_drivers": "Nenhum condutor conhecido nas proximidades.",
  "im_here_btn": "Estou Aqui! · Notificar Rede",
  "at_stop": "Na paragem"
  // Todas as restantes strings herdadas de pt-AO
}

```

### 3.3 Configuração do i18next com herança

src/i18n/index.ts

```

// Ficheiro: index.ts | Função: Configuração i18next com herança por mercado
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import { detectMarket } from './detector';

```

```
// Importar todos os locais
import ptAO_common from './locales/pt-AO/common.json';
import ptAO_ride from './locales/pt-AO/ride.json';
import ptAO_auth from './locales/pt-AO/auth.json';
import ptAO_errors from './locales/pt-AO/errors.json';

import ptBR_ride from './locales/pt-BR/ride.json';
import ptBR_auth from './locales/pt-BR/auth.json';

import ptPT_ride from './locales/pt-PT/ride.json';

import enNG_common from './locales/en-NG/common.json';
import enNG_ride from './locales/en-NG/ride.json';
import enNG_auth from './locales/en-NG/auth.json';

const resources = {
  // Angola - base (outros herdam daqui)
  'pt-AO': { common: ptAO_common, ride: ptAO_ride,
    auth: ptAO_auth, errors: ptAO_errors },
  // Moçambique - herda de AO, sem diferenças por agora
  'pt-MZ': { ride: {}, auth: {} },
  // Brasil - sobrescreve termos específicos
  'pt-BR': { ride: ptBR_ride, auth: ptBR_auth },
  // Cabo Verde - herda de AO, sem diferenças por agora
  'pt-CV': { ride: {}, auth: {} },
  // Portugal - sobrescreve condutor/condutores
  'pt-PT': { ride: ptPT_ride },
  // Nigéria - inglês completo, sem herança
  'en-NG': { common: enNG_common, ride: enNG_ride, auth: enNG_auth },
};

export async function initI18n() {
  const market = await detectMarket();
  const marketToLocale: Record<string, string> = {
    ao: 'pt-AO', mz: 'pt-MZ', br: 'pt-BR',
    cv: 'pt-CV', pt: 'pt-PT', ng: 'en-NG',
  };
  const locale = marketToLocale[market];

  await i18n.use(initReactI18next).init({
    resources,
    lng: locale,
    fallbackLng: 'pt-AO', // herança para todos os pt-XX
    ns: ["common", "ride", "auth", "errors"],
    defaultNS: 'common',
    interpolation: { escapeValue: false },
  });
}
```

### 3.4 Hook de tradução com autocomplete TypeScript

```
src/hooks/useTranslation.ts
```

```
// Ficheiro: useTranslation.ts | Função: Wrapper tipado do i18next
import { useTranslation as useI18n } from 'react-i18next';

// Uso: const { t, market } = useT("ride");
// t("request_ride") → "Pedir Boleia" (AO) ou "Pedir Carona" (BR)
export function useT(namespace: 'common' | 'ride' | 'auth' | 'errors' = 'common')
{
  const { t, i18n } = useI18n(namespace);
  return {
    t,
    locale: i18n.language,
    changeLocale: (locale: string) => i18n.changeLanguage(locale),
  };
}
```

## 4. Comparação Completa por Mercado

| Dimensão    | 🇦🇴 Angola         | 🇲🇐 Moçambique   | 🇧🇷 Brasil         | 🇨🇻 Cabo Verde  | 🇵🇹 Portugal         | 🇳🇬 Nigéria             |
|-------------|-------------------|-----------------|-------------------|----------------|---------------------|------------------------|
| Idioma      | pt-AO             | pt-MZ           | pt-BR             | pt-CV          | pt-PT               | en-NG                  |
| Moeda       | AOA (Kz)          | MZN (MT)        | BRL (R\$)         | CVE (Esc)      | EUR (€)             | NGN (₦)                |
| Telefone    | +244              | +258            | +55               | +238           | +351                | +234                   |
| SMS OTP     | AfricasTalking    | AfricasTalking  | Twilio            | AfricasTalking | Twilio              | Termii                 |
| Mapas       | OSM               | OSM             | Google            | OSM            | Google              | OSM                    |
| Pagamentos  | Multicaixa / EMIS | M-Pesa / e-Mola | PIX / Pague menos | Vinti4         | MB Way / Multibanco | Paystack / Flutterwave |
| "Boleia"    | Boleia            | Boleia          | Carona            | Boleia         | Boleia              | Lift / Ride            |
| "Paragem"   | Paragem           | Paragem         | Ponto de Ônibus   | Paragem        | Paragem             | Bus Stop               |
| "Motorista" | Motorista         | Motorista       | Motorista         | Motorista      | Condutor            | Driver                 |
| "Agora!"    | Agora!            | Agora!          | Tô aqui!          | Agora!         | Estou aqui!         | I'm Here!              |
| Emergência  | 113               | 119             | 190               | 132            | 112                 | 199                    |
| Privacidade | —                 | —               | LGPD              | —              | RGPD                | —                      |

### 4.1 Notas por mercado

#### 🇦🇴 Angola (ao)

Mercado primário. AfricasTalking integrado nativamente com Unitel e Africell.

#### 🇲🇐 Moçambique (mz)

Identical language to AO. Diferencia-se em moeda, operadores móveis e pagamentos.

#### 🇧🇷 Brasil (br)

Vocabulário diferente: "carona" em vez de "boleia". Concordância verbal brasileira.

#### 🇨🇻 Cabo Verde (cv)

Menor mercado mas insulano — lógica de percurso diferente (viagens inter-ilhas não relevantes).

#### 🇵🇹 Portugal (pt)

Mercado mais regulamentado — RGPD muito mais exigente. Uso urbano + rural (interior).

#### 🇳🇬 Nigéria (ng)

Inglês nigeriano. Grande mercado — Lagos sozinha tem 15M habitantes. Parceiro local recomendado.

## 5. Selector de Mercado na App

Quando a app não consegue detectar o mercado automaticamente, ou quando o utilizador quer mudar, aparece o ecrã de selecção de mercado. Este ecrã também aparece como opção nas Definições.

### 5.1 Componente de selecção

```
src/screens/auth/MarketSelectScreen.tsx (simplificado)

// Ficheiro: MarketSelectScreen.tsx | Função: Selecção manual de mercado
import React from 'react';
import { View, FlatList, TouchableOpacity, Text, StyleSheet } from 'react-native';
import { setMarket } from '@il8n/detector';
import { loadMarketConfig } from '@config/markets';
import { useMarketStore } from '@store/marketStore';
import type { MarketCode } from '@config/markets/types';

const MARKETS = [
  { code: 'ao', flag: '🇦🇴', name: 'Angola', sub: 'Luanda · Huambo · Benguela' },
  { code: 'mz', flag: '🇲🇿', name: 'Moçambique', sub: 'Maputo · Beira · Nampula' },
  { code: 'br', flag: '🇧🇷', name: 'Brasil', sub: 'São Paulo · Rio · Recife' },
  { code: 'cv', flag: '🇨🇻', name: 'Cabo Verde', sub: 'Praia · Mindelo' },
  { code: 'pt', flag: '🇵🇹', name: 'Portugal', sub: 'Lisboa · Porto · Interior' },
  { code: 'ng', flag: '🇳🇬', name: 'Nigeria', sub: 'Lagos · Abuja · PH' },
] as const;

export function MarketSelectScreen({ onSelect }) {
  async function handleSelect(code: MarketCode) {
    await setMarket(code); // guarda escolha
    const config = await loadMarketConfig(code); // carrega config
    useMarketStore.getState().setMarket(config); // actualiza store
    onSelect(code);
  }

  return (
    <FlatList
      data={MARKETS}
      keyExtractor={m => m.code}
      renderItem={({ item: m }) => (
        <TouchableOpacity onPress={() => handleSelect(m.code as MarketCode)}
          style={styles.row}>
          <Text style={styles.flag}>{m.flag}</Text>
          <View>
            <Text style={styles.name}>{m.name}</Text>
            <Text style={styles.sub}>{m.sub}</Text>
          </View>
        </TouchableOpacity>
      )}
    />
  );
}
```

```
        </View>
      </TouchableOpacity>
    )}
  />
);
}
```

## 5.2 useMarket — hook central de configuração

```
src/hooks/useMarket.ts

// Ficheiro: useMarket.ts | Função: Acesso a toda a configuração do mercado
activo
import { useMarketStore } from '@store/marketStore';

// Uso em qualquer componente:
// const { config, t, formatCurrency, formatPhone } = useMarket();
export function useMarket() {
  const config = useMarketStore(s => s.config);

  // Formata valor monetário conforme a moeda do mercado
  function formatCurrency(amount: number): string {
    return `${config.currency.symbol}
    ${amount.toFixed(config.currency.decimals)}`;
    // AO: "Kz 1500" | BR: "R$ 12,50" | PT: "€ 3,20"
  }

  // Valida formato de telemóvel do mercado
  function validatePhone(phone: string): boolean {
    const digits = phone.replace(/\D/g, "");
    return digits.length >= config.phoneMinDigits &&
      digits.length <= config.phoneMaxDigits;
  }

  // Formata telemóvel para exibição
  function formatPhone(phone: string): string {
    return `${config.phonePrefix} ${phone}`;
  }

  // Verifica se uma feature está activa neste mercado
  function hasFeature(feature: keyof typeof config.features): boolean {
    return config.features[feature] ?? false;
  }

  return { config, formatCurrency, validatePhone, formatPhone, hasFeature };
}
```

## 6. Geocoding, Ruas e Paragens por Mercado

O nome das ruas, bairros e paragens muda drasticamente de mercado para mercado — não só o idioma mas a própria estrutura de endereços. Em Luanda existe o "Bairro Operário". Em São Paulo existe a "Avenida Paulista". Em Lagos existe "Victoria Island". O sistema de geocoding precisa de adaptar-se a estas realidades.

### 6.1 Serviço de geocoding adaptativo

```
src/services/geocoding.service.ts

// Ficheiro: geocoding.service.ts | Função: Geocoding adaptado ao mercado activo
import { useMarketStore } from '@store/marketStore';

interface GeoResult {
  displayName: string // ex: "Av. Lenine, Ingombota, Luanda"
  shortName: string // ex: "Ingombota · P.4" (para mostrar na app)
  lat: number
  lng: number
}

export async function reverseGeocode(lat: number, lng: number):
Promise<GeoResult> {
  const config = useMarketStore.getState().config;

  if (config.geocodingProvider === 'google') {
    return reverseGeocodeGoogle(lat, lng);
  }
  return reverseGeocodeNominatim(lat, lng);
}

async function reverseGeocodeNominatim(lat: number, lng: number):
Promise<GeoResult> {
  const url = `https://nominatim.openstreetmap.org/reverse`
    + `?lat=${lat}&lon=${lng}&format=json&addressdetails=1`
    + `&accept-language=${useMarketStore.getState().config.locale}`;

  const res = await fetch(url, {
    headers: { 'User-Agent': 'RideFriend/1.0 (suporte@ridefriend.ao)' }
  });
  const data = await res.json();

  const addr = data.address;
  // Constrói nome curto adaptado a cada estrutura de endereço
  const neighbourhood = addr.neighbourhood || addr.suburb || addr.quarter || "";
  const road = addr.road || addr.pedestrian || addr.path || "";
  const shortName = neighbourhood
    ? `${neighbourhood}${road ? " · " + road : ""}`
    : road || data.display_name.split(",")[0];

  return {
```



```

    displayName: data.display_name,
    shortName,
    lat,
    lng,
  };
}

// Autocomplete de destino — usa Nominatim com bias de localização
export async function searchAddress(query: string, nearLat: number, nearLng:
number) {
  const config = useMarketStore.getState().config;
  const url = `https://nominatim.openstreetmap.org/search`
    + `?q=${encodeURIComponent(query)}&format=json&limit=5`
    + `&viewbox=${nearLng-0.3},${nearLat-0.3},${nearLng+0.3},${nearLat+0.3}`
    + `&bounded=0&addressdetails=1`
    + `&accept-language=${config.locale}`;

  const res = await fetch(url, {
    headers: { 'User-Agent': 'RideFriend/1.0' }
  });
  return res.json();
}

```

## 6.2 Paragens pré-carregadas por mercado (base de dados)

Para as cidades principais de cada mercado, a app tem um conjunto de paragens de referência pré-carregadas na base de dados. Isto permite sugerir paragens ao utilizador mesmo sem ligação à internet, e torna os alertas de proximidade mais precisos.

```

supabase/migrations/stops_seed.sql (exemplo)

-- Tabela de paragens de referência por mercado e cidade
CREATE TABLE bus_stops (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  market_code VARCHAR(2) NOT NULL, -- "ao", "mz", "br", etc.
  city        VARCHAR(50) NOT NULL,
  name        VARCHAR(100) NOT NULL, -- nome da paragem
  lat         DECIMAL(10,8) NOT NULL,
  lng         DECIMAL(11,8) NOT NULL,
  is_major    BOOLEAN DEFAULT false -- paragem principal vs secundária
);

-- Índice geográfico para queries de proximidade
CREATE INDEX stops_geo_idx ON bus_stops USING GIST (
  ST_SetSRID(ST_MakePoint(lng, lat), 4326)
);

-- Dados Angola (Luanda)
INSERT INTO bus_stops (market_code, city, name, lat, lng, is_major) VALUES
('ao','Luanda','Av. Lenine · Paragem Central', -8.8147, 13.2302, true),

```

```

    ('ao','Luanda','Largo do Kinaxixe',          -8.8200, 13.2350, true),
    ('ao','Luanda','Talatona · Shopping',        -8.9186, 13.1847, true),
    ('ao','Luanda','Viana · Paragem Principal',   -8.9039, 13.3728, true),

-- Dados Brasil (São Paulo)
    ('br','São Paulo','Terminal Jabaquara',      -23.6427, -46.6534, true),
    ('br','São Paulo','Av. Paulista c/ Consolação', -23.5614, -46.6565, true),
    ('br','São Paulo','Terminal Capelinha',      -23.6669, -46.7670, true),

-- Dados Portugal (Lisboa)
    ('pt','Lisboa','Marquês de Pombal',          38.7252,  -9.1500, true),
    ('pt','Lisboa','Oriente · Gare do Oriente',  38.7681,  -9.0990, true),

-- Dados Nigéria (Lagos)
    ('ng','Lagos','Oshodi Bus Terminal',         6.5480,  3.3540, true),
    ('ng','Lagos','CMS Bus Stop',                 6.4536,  3.3947, true);

```

## 6.3 Função: paragem mais próxima do utilizador

```

src/services/stops.service.ts

// Ficheiro: stops.service.ts | Função: Encontra paragem mais próxima
import { supabase } from './supabase';
import { useMarketStore } from '@store/marketStore';

export async function getNearestStop(lat: number, lng: number) {
  const { code } = useMarketStore.getState().config;

  const { data, error } = await supabase.rpc("nearest_stop", {
    p_lat: lat,
    p_lng: lng,
    p_market: code,
    p_radius_m: 800 // raio de 800m
  });

  if (error || !data?.length) return null;
  return data[0]; // { name, lat, lng, distance_m }
}

```

## 7. Sistema SMS Multi-Provider

Cada mercado tem o seu melhor gateway de SMS. Em Angola e Moçambique, o AfricasTalking tem acordos directos com as operadoras locais. No Brasil, o Twilio tem cobertura total. Na Nigéria, o Termii é o mais usado por startups. O sistema de SMS do RideFriend detecta automaticamente qual provider usar.

```
src/services/sms.service.ts - dispatcher multi-provider
// Ficheiro: sms.service.ts | Função: Envio de SMS adaptado ao mercado
import { useMarketStore } from '@store/marketStore';

export async function sendOtpSms(phone: string, code: string): Promise<void> {
  const { smsProvider, locale } = useMarketStore.getState().config;

  // Mensagem adaptada ao idioma do mercado
  const messages: Record<string, string> = {
    'pt-AO': `O teu código RideFriend é: ${code}. Válido 10 min.`,
    'pt-MZ': `O teu código RideFriend é: ${code}. Válido 10 min.`,
    'pt-BR': `Seu código RideFriend é: ${code}. Válido por 10 min.`,
    'pt-CV': `O teu código RideFriend é: ${code}. Válido 10 min.`,
    'pt-PT': `O teu código RideFriend é: ${code}. Válido 10 min.`,
    'en-NG': `Your RideFriend code is: ${code}. Valid for 10 minutes.`,
  };

  const message = messages[locale] || messages["pt-AO"];

  switch (smsProvider) {
    case 'africas_talking': return sendAfricasTalking(phone, message);
    case 'twilio':          return sendTwilio(phone, message);
    case 'termii':          return sendTermii(phone, message);
    default:                throw new Error(`Provider desconhecido: ${smsProvider}`);
  }
}

async function sendAfricasTalking(phone: string, message: string) {
  // AfricasTalking - Angola, Moçambique, Cabo Verde, Nigéria
  const res = await fetch("https://api.africastalking.com/version1/messaging", {
    method: 'POST',
    headers: { 'apiKey': process.env.AT_API_KEY!, 'Content-Type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({ username: process.env.AT_USERNAME!, to: phone, message }),
  });
  if (!res.ok) throw new Error("Falha ao enviar SMS via AfricasTalking");
}

async function sendTwilio(phone: string, message: string) {
  // Twilio - Brasil, Portugal
  const auth =
    Buffer.from(`${process.env.TWILIO_SID}:${process.env.TWILIO_TOKEN}`).toString("base64");
```

```
    await fetch(`https://api.twilio.com/2010-04-01/Accounts/${process.env.TWILIO_SID}/Messages.json`, {
      method: 'POST',
      headers: { Authorization: `Basic ${auth}` },
      body: new URLSearchParams({ From: process.env.TWILIO_FROM!, To: phone, Body: message }),
    });
  }

  async function sendTermii(phone: string, message: string) {
    // Termii - Nigéria
    await fetch("https://api.ng.termii.com/api/sms/send", {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        api_key: process.env.TERMII_API_KEY,
        to: phone, from: 'RideFriend',
        sms: message, type: 'plain', channel: 'generic'
      }),
    });
  }
}
```

## 8. Prompt de Implementação — Localização Multi-Mercado

Use este prompt no Claude Project RideFriend Dev para implementar toda a arquitetura de localização descrita neste documento.

**PRÉ-REQUISITO:** Este prompt deve ser usado APÓS o Prompt 1 (setup inicial). A estrutura de pastas base deve já existir.

### PROMPT L1 — Arquitetura i18n + Market Config (cole no Claude Project)

Implementa a arquitetura completa de multi-mercado para o RideFriend. Suportar: Angola (ao), Moçambique (mz), Brasil (br), Cabo Verde (cv), Portugal (pt), Nigéria (ng). Uma única codebase para todos.

Instalar dependências:

```
npm install i18next react-i18next expo-localization
```

- src/config/markets/types.ts  
Interface MarketConfig completa com todos os campos:  
code, name, locale, timezone, phonePrefix, phoneMin/MaxDigits, phonePlaceholder, smsProvider, mapsProvider, defaultCenter, defaultZoom, geocodingProvider, currency{code,symbol,decimals}, emergencyNumber, privacyRegulation, dataResidency, paymentMethods?, features{...}, theme{accentColor,accentColorLight}, supportWhatsApp?, appId?  
Type MarketCode = "ao"|"mz"|"br"|"cv"|"pt"|"ng"
- src/config/markets/ao.config.ts (Angola — REFERÊNCIA BASE)  
Todos os campos preenchidos. Valores Angola conforme especificação.
- src/config/markets/mz.config.ts (Moçambique)  
Diferenças: +258, MZN, Africa/Maputo, M-Pesa, emergência 119
- src/config/markets/br.config.ts (Brasil)  
Diferenças: +55, BRL, America/Sao\_Paulo, Twilio, Google Maps, PIX, features.inAppPayments=true, features.gdprConsent=false, privacyRegulation="lgpd", emergência 190
- src/config/markets/cv.config.ts (Cabo Verde)  
Diferenças: +238, CVE, Atlantic/Cape\_Verde, Vinti4, emergência 132
- src/config/markets/pt.config.ts (Portugal)  
Diferenças: +351, EUR, Europe/Lisbon, Twilio, Google Maps, MB Way, features.gdprConsent=true, privacyRegulation="rgpd", dataResidency="eu", emergência 112
- src/config/markets/ng.config.ts (Nigéria)  
Diferenças: +234, NGN, Africa/Lagos, en-NG, Termii/AfricasTalking, Paystack, emergência 199, locale="en-NG"

```
8. src/config/markets/index.ts
  Função loadMarketConfig(code): Promise<MarketConfig>
  Importa e retorna a config do mercado pedido

9. src/il8n/detector.ts
  detectMarket(): Promise<MarketCode>
  - 1º: AsyncStorage rf_market (escolha guardada)
  - 2º: Localization.locale do dispositivo
  - 3º: Fallback "ao"
  setMarket(code): Promise<void>

10. src/il8n/locales/pt-AO/common.json (base completo)
    src/il8n/locales/pt-AO/ride.json (termos de boleia)
    src/il8n/locales/pt-AO/auth.json (ecrãs de auth)
    src/il8n/locales/pt-AO/errors.json (mensagens de erro)

11. src/il8n/locales/pt-BR/ride.json (só diferenças: carona, ônibus)
    src/il8n/locales/pt-BR/auth.json
    src/il8n/locales/pt-PT/ride.json (só: condutor, condutores)
    src/il8n/locales/en-NG/common.json (inglês completo)
    src/il8n/locales/en-NG/ride.json
    src/il8n/locales/en-NG/auth.json

12. src/il8n/index.ts
    initI18n(): Promise<void>
    - i18next + initReactI18next
    - fallbackLng: "pt-AO" para herança de todos os pt-XX
    - Carregar todos os locales como resources

13. src/store/marketStore.ts (Zustand)
    Estado: config: MarketConfig, isLoading: boolean
    Acções: setMarket(config), loadFromDetector()

14. src/hooks/useMarket.ts
    Expõe: config, formatCurrency, validatePhone,
    formatPhone, hasFeature

15. src/hooks/useT.ts
    Wrapper tipado do useTranslation do react-i18next

16. src/screens/auth/MarketSelectScreen.tsx
    Lista dos 6 mercados com bandeira, nome e cidades principais
    onSelect(code) chama setMarket + loadMarketConfig + navigation

17. src/services/sms.service.ts
    sendOtpSms(phone, code): Promise<void>
    Dispatcher: african_talking | twilio | termii
    Mensagem OTP adaptada ao locale do mercado

18. src/services/geocoding.service.ts
    reverseGeocode(lat, lng): Promise<GeoResult>
    searchAddress(query, nearLat, nearLng): Promise<GeoResult[]>
```

Usa Nominatim ou Google conforme mapsProvider da config

Ao final: mostra como usar em qualquer componente:

```
const { t } = useT("ride");
const { config, hasFeature } = useMarket();
t("request_ride") // → "Pedir Boleia" (AO) ou "Pedir Carona" (BR)
```

#### PROMPT L2 – Ecrã de Selecção de Mercado no Onboarding

O ecrã de onboarding deve incluir a selecção de mercado.

Modifica `src/screens/auth/OnboardingScreen.tsx` para:

- Mostrar passo de selecção de mercado ANTES do formulário de perfil
  - Lista dos 6 mercados com bandeira + nome + cidades
  - Destaque no mercado detectado automaticamente
  - Texto "Detectamos que estás em Angola. Correcto?" com opção de mudar
- Após selecção de mercado:
  - Chamar `setMarket(code) + loadMarketConfig(code)`
  - Actualizar o `i18n` para o locale do mercado
  - Actualizar o tema da app (`accentColor` do mercado)
- No formulário de perfil:
  - Placeholder do telefone adapta-se ao mercado  
AO: "9XX XXX XXX" | BR: "(11) 9XXXX-XXXX" | NG: "080X XXX XXXX"
  - Label do campo "Bairro/Zona" adapta-se:  
AO/MZ: "Bairro" | BR: "Bairro" | PT: "Localidade" | NG: "Area"
- Opção nas Definições (`SettingsScreen`) para mudar de mercado:
  - "Mudar de País" → abre `MarketSelectScreen`
  - Aviso: "Mudar o país redefine as configurações locais"

#### PROMPT L3 – Feature Flags por Mercado

Implementa o sistema de feature flags no RideFriend.

- Criar `src/config/features.ts` com a função:
 

```
isEnabled(feature: FeatureFlag, market: MarketConfig): boolean
```
- Features a implementar:
  - `inAppPayments`: false em AO/MZ/CV/PT/NG, true só no Brasil (PIX)
    - Quando true: mostrar botão "Pagar com PIX" no ecrã de confirmação
    - Quando false: esconder completamente a secção de pagamento
  - `gdprConsent`: false em todos excepto PT
    - Quando true: mostrar ecrã de consentimento RGPD no primeiro launch
    - Incluir opções: aceitar, recusar, ver política completa
  - `vehicleVerification`: false em AO/MZ/CV, true em BR/PT/NG

→ Quando true: adicionar campo de verificação de matrícula no onboarding

googleMaps: false em AO/MZ/CV/NG, true em BR/PT

→ Quando true: usar PROVIDER\_GOOGLE em react-native-maps

→ Quando false: usar PROVIDER\_DEFAULT com tiles OpenStreetMap

3. Usar o hook useMarket().hasFeature() em todos os componentes:

```
const { hasFeature } = useMarket();  
{hasFeature("inAppPayments") && <PaymentButton />}  
{hasFeature("gdprConsent") && <GdprBanner />}
```



## 9. Sumário de Implementação

### 9.1 O que é código universal (não tocar para novos mercados)

- Algoritmo Haversine, cálculo de rota, detecção de passageiros
- Lógica de estados da boleia (requested → accepted → completed)
- Sistema de ratings, reputação, historial
- Notificações push (Expo Notifications — mesmo código, mensagens via i18n)
- Componentes UI (mudam cor via theme, mudam texto via i18n)
- Base de dados Supabase (mesmo schema, dados separados por market\_code)
- Toda a navegação e estrutura de ecrãs

### 9.2 O que muda para adicionar um novo mercado

Para adicionar um 7.º mercado (ex: Kenya 🇰🇪), apenas é necessário:

1. Criar src/config/markets/ke.config.ts com os dados do Kenya
2. Adicionar "ke" ao tipo MarketCode em types.ts
3. Criar src/i18n/locales/en-KE/ com as traduções (herda de en-NG)
4. Adicionar o Kenya à lista em MarketSelectScreen.tsx
5. Adicionar paragens de Nairobi em bus\_stops (SQL)
6. Configurar conta AfricasTalking para Kenya (+254)

Estimativa de trabalho para adicionar um novo mercado: 2-4 horas.

*Conclusão: A arquitectura de market config + i18n com herança significa que 95% do trabalho é feito uma vez. Cada novo mercado requer apenas configuração e dados — nunca reescrever código.*